

Contents

Run Guide - PMSM Fault Diagnosis Pipeline

0. The one thing that differs between operating systems

1. Prerequisites

1.1 Python 3.10, 3.11, or 3.12 - required

1.2 Git - required to clone the project

1.3 Optional extras

2. Get the code and create the virtual environment

2.1 Clone

2.2 Create the venv

3. Activate the environment (do this in every new terminal)

4. Install dependencies

5. Quickstart - verify it works (any OS, venv active)

Interactive dashboard

6. Running each stage individually

7. Using real data (KAIST PMSM dataset)

8. Optional: GPU acceleration (NVIDIA only)

9. Optional: rebuilding the reports/slides (PDF, Word, PowerPoint)

10. Troubleshooting

11. Daily workflow cheat-sheet

Run Guide - PMSM Fault Diagnosis Pipeline

A complete, copy-paste guide to running this project on Windows, macOS, and Linux.

This guide assumes **no prior setup**. Follow it top to bottom the first time; afterwards you only need \$3 (activate) and \$5/\$6 (run).

What you are running. A Python pipeline that turns motor current/vibration signals into **wavelet scalogram images** and classifies motor faults with a **CNN**. Everything runs in pure Python - **MATLAB is not required**. There is also an interactive **Streamlit dashboard** that walks through the whole project.

0. The one thing that differs between operating systems

99% of this project is identical on every OS. The **only** real difference is how you create and activate the Python virtual environment, and the path to the interpreter inside it:

	Linux / macOS	Windows (PowerShell)	Windows (CMD)	Git Bash (Windows)
Activate venv	source .venv/bin/activate	.venv\Scripts\Activate.ps1	.venv\Scripts\activate.bat	.venv/Scripts/activate
Interpreter path	.venv/bin/python	venv\Scripts\python	venv\Scripts\python	venv/Scripts/python

	Linux / macOS	Windows (PowerShell)	Windows (CMD)	Git Bash (Windows)
make available?	yes (built-in / apt/brew)	no (use raw commands, §6)	no	only if installed

The simplest cross-platform strategy (recommended): *activate the venv once* (§3), after which plain python and pip refer to the venv on every OS. Then you never type the long `.venv/...` paths again. This guide uses activated commands by default and shows the explicit paths only where it matters.

1. Prerequisites

1.1 Python 3.10, 3.11, or 3.12 - required

TensorFlow has no wheel for Python 3.13 or 3.14 yet, so you *must* use 3.10-3.12. Check what you have:

```
python3 --version      # Linux/macOS
python --version       # Windows
```

If it prints 3.10, 3.11, or 3.12 you are done. Otherwise install one:

- **Windows** - download the 3.12 installer from <https://www.python.org/downloads/>. **IMPORTANT: tick “Add python.exe to PATH”** on the first installer screen. (Or, if you use winget: `winget install Python.Python.3.12.`)
- **macOS** - brew install `python@3.12` (needs [Homebrew](#)), or the python.org installer.
- **Linux (Debian/Ubuntu) -**

```
sudo apt update
sudo apt install -y python3.10 python3.10-venv python3-pip git
```

(Fedora: `sudo dnf install python3.10`; Arch: `sudo pacman -S python git.`)

If you have several Pythons installed, call the exact one when creating the venv, e.g. `py -3.12` on Windows, or `python3.10` on Linux/macOS. See §2.

1.2 Git - required to clone the project

- Windows: `winget install Git.Git` (this also gives you **Git Bash**).
- macOS: brew install `git` (or it ships with the Xcode command-line tools).
- Linux: `sudo apt install git`.

1.3 Optional extras

- **make** - lets you use the short `make ...` targets. Built into macOS (with Xcode CLT) and Linux. On Windows it is *not* required - §6 gives the raw command for every target. (If you want it on Windows: `winget install GnuWin32.Make`, or use it inside WSL/Git Bash.)

- **NVIDIA GPU + CUDA** - optional, only speeds up training. See §8.
 - **pandoc + xelatex** - only needed if you want to *rebuild the PDF/Word reports* yourself. See §9. Not needed to run the pipeline.
-

2. Get the code and create the virtual environment

2.1 Clone

```
git clone https://github.com/molhamfetnah/pmsm-fault-diagnosis-cnn-scalogram.git
cd pmsm-fault-diagnosis-cnn-scalogram
```

(If you already have the folder, just cd into it.)

2.2 Create the venv

A *virtual environment* keeps this project's packages isolated from the rest of your system. Create it **once**:

Linux / macOS:

```
python3.10 -m venv .venv      # or python3.11 / python3.12
```

Windows (PowerShell or CMD):

```
py -3.12 -m venv .venv      # 'py' is the Python launcher installed with Python
```

(If py isn't found, use python -m venv .venv - assuming your python is 3.10-3.12.)

This creates a .venv/ folder. You do **not** commit it; it's git-ignored.

3. Activate the environment (do this in every new terminal)

Activation makes python/pip point at the venv. You must re-activate each time you open a new terminal.

Linux / macOS (bash/zsh):

```
source .venv/bin/activate
```

Windows - PowerShell:

```
.venv\Scripts\Activate.ps1
```

First time only, if PowerShell blocks the script with an “*execution policy*” error, run this once and try again:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

Windows - CMD:

```
.venv\Scripts\activate.bat
```

Windows - Git Bash:

```
source .venv/Scripts/activate
```

When active, your prompt shows (.venv). To leave it later: deactivate.

4. Install dependencies

With the venv **active**:

```
python -m pip install --upgrade pip
pip install -r requirements.txt
```

This installs TensorFlow, NumPy, SciPy, scikit-learn, PyWavelets, pandas, matplotlib, npT-DMS (for reading KAIST .tdms files), and pytest. CPU-only build by default - that is fine; it just trains a bit slower than a GPU.

For the interactive dashboard, also install:

```
pip install -r requirements-dashboard.txt
```

Windows note: all these packages ship as pre-built wheels, so no C++ compiler is needed. If pip is slow on the large TensorFlow download, just let it finish.

5. Quickstart - verify it works (any OS, venv active)

These three commands are the fastest way to confirm a healthy install. On Windows (no make), use the right-hand “raw command” from §6.

```
make test      # run the unit-test suite - should report all passed
make demo      # synthetic end-to-end: simulate -> scalograms -> train -> evaluate
```

make demo generates synthetic motor signals, renders scalograms, trains a small CNN, and writes: - a trained model to models/cnn_current.keras - a confusion matrix + metrics to results/

If that completes without error, **your setup is correct**.

Interactive dashboard

```
# Linux/macOS:
./run_demo.sh
# Any OS (works everywhere, venv active):
python -m streamlit run app.py
```

It opens at <http://localhost:8501>. The dashboard has nine sections (Pipeline, The Problem, Signal Lab, Scalogram Studio, Dataset Explorer, The CNN Model, Results & Ablations, **Test Lab** - test the model by hand - and Concepts & Defense Prep). It works even with no data/model present (the educational and synthetic parts always run). Press Ctrl+C in the terminal to stop it.

6. Running each stage individually

Two ways to call everything. **Pick the column for your OS.** Both do exactly the same thing - make is just a shortcut for the raw command.

Step	With make (Linux/macOS)	Raw command (any OS, venv active)
Run tests	<code>make test</code>	<code>python -m pytest -q</code>
Generate synthetic signals	<code>make simulate</code>	<code>python -m python.simulate</code>
Render CWT scalograms	<code>make scalograms</code>	<code>python -m python.scalogram</code>
Leakage-free train/val/test split	<code>make split</code>	<code>python -m python.run_split</code>
Train CNN (current)	<code>make train SIGNAL=current EPOCHS=20</code>	<code>python -m python.train --signal-type current --epochs 20 --arch baseline</code>
Train CNN (vibration)	<code>make train SIGNAL=vibration</code>	<code>python -m python.train --signal-type vibration --epochs 20</code>
Evaluate on test set	<code>make evaluate SIGNAL=current</code>	<code>python -m python.evaluate --signal-type current</code>
Dual-branch fusion model	<code>make train-fusion</code>	<code>python -m python.train_fusion --epochs 20</code>
Consolidated results summary	<code>make report</code>	<code>python -m python.report_metrics</code>
Generated-data study	<code>make transfer</code>	<code>python -m python.transfer_experiment</code>

Windows users: if you did *not* activate the venv (§3), prefix python with the full path: `.venv\Scripts\python.exe -m python.simulate`, etc. Activating is simpler.

Architecture choice (`--arch`): `baseline` (default), `modern` (BatchNorm + global-average-pooling), or `transfer` (frozen MobileNetV2 on ImageNet - the strongest on the small current set). Example:

```
python -m python.train --signal-type current --epochs 20 --arch transfer
```

All parameters (window length, sampling rate, image size, wavelet, number of scales, class list, paths...) live in **config.yaml** - edit there, not in code. `data/manifest.csv` is the single source of truth linking every signal segment -> its scalogram -> its label and split.

7. Using real data (KAIST PMSM dataset)

The headline results come from the real **KAIST** dataset (current @ 100 kHz, vibration @ 25.6 kHz, .tdms format).

1. Download the .tdms files from [Mendeley rgn5brrgrn](https://doi.org/10.17632/rgn5brrgrn.5) (DOI 10.17632/rgn5brrgrn.5, CC-BY-4.0) into **data/raw/mendeley_pmsm/**.
2. Ingest, then run the pipeline (venv active):

```
python -m python.ingest_mendeley      # decimates 100 kHz -> config target_fs
make scalograms split                 # or the raw commands from §6
make train SIGNAL=vibration && make evaluate SIGNAL=vibration
make train SIGNAL=current && make evaluate SIGNAL=current
```

Other dataset loaders are also ready (ingest_ieee_pmsm.py, ingest_uottawa.py); see docs/data-audit.md and docs/data-expansion.md for sources and status.

8. Optional: GPU acceleration (NVIDIA only)

The default install is **CPU-only and works fine**. To use an NVIDIA GPU (verified ~17x faster per step on a Quadro P620):

```
pip install "tensorflow[and-cuda]==2.16.*"
```

TensorFlow then auto-detects the card - no code change. On a small-VRAM GPU, stop TF from grabbing all memory at once:

```
# Linux/macOS:
export TF_FORCE_GPU_ALLOW_GROWTH=true
# Windows PowerShell:
$env:TF_FORCE_GPU_ALLOW_GROWTH = "true"
# Windows CMD:
set TF_FORCE_GPU_ALLOW_GROWTH=true
```

Verify TF sees the GPU:

```
python -c "import tensorflow as tf; print(tf.config.list_physical_devices('GPU'))"
```

The dashboard deliberately runs on CPU (it sets CUDA_VISIBLE_DEVICES=-1) so it never competes with a training job for GPU memory.

9. Optional: rebuilding the reports/slides (PDF, Word, PowerPoint)

Only needed if you edit the markdown sources in docs/ and want fresh documents. Requires **pandoc** and **xelatex**, plus the **Amiri** font for the Arabic PDFs.

- Linux: `sudo apt install pandoc texlive-xetex texlive-fonts-extra`
- macOS: `brew install pandoc + brew install --cask mactex (or basictex)`
- Windows: `winget install JohnMacFarlane.Pandoc + install MiKTeX`; then run the build inside Git Bash/WSL (the `make docs` recipe uses Unix shell syntax).

```
make docs      # English: report, engineering-background, slides, guides -> docs/build
make docs-ar   # Arabic (RTL) versions
```

Output lands in docs/build/ as .pdf, .docx, and .pptx.

10. Troubleshooting

Symptom	Cause & fix
Could not find a version that satisfies tensorflow PowerShell: <i>"running scripts is disabled on this system"</i>	You're on Python 3.13/3.14. Recreate the venv with 3.10-3.12 (§1.1, §2.2). Run Set-ExecutionPolicy -Scope CurrentUser RemoteSigned once, then activate again (§3).
make: command not found (Windows)	Use the raw commands in the right-hand column of §6 - make is optional.
python opens the Microsoft Store (Windows)	The Store alias is intercepting it. Use py -3.12 . . . , or disable the alias in <i>Settings -> Apps -> App execution aliases</i> .
ModuleNotFoundError: No module named 'python'	Run from the project root (where config.yaml is), and use python -m python.<module> (note the -m), not python python/<module>.py.
ModuleNotFoundError: tensorflow / others	The venv isn't active, or deps aren't installed. Re-do §3 and §4.
Training is very slow	Normal on CPU. Lower EP0CHS, or set up the GPU (§8).
TF prints CUDA / GPU warnings on CPU-only machine	Harmless - TF just notes it found no GPU and uses the CPU.
Dashboard won't open / port in use	Another app uses 8501: python -m streamlit run app.py --server.port 8502.
npTDMS / .tdms read errors	You're pointing at non-KAIST files, or they're not in data/raw/mendeley_pmsm/. See §7.

11. Daily workflow cheat-sheet

```
# every new terminal:
cd pmsm-fault-diagnosis-cnn-scalogram
source .venv/bin/activate      # Windows: .venv\Scripts\Activate.ps1

# then any of:
make test                      # or: python -m pytest -q
make demo                      # full synthetic run
```

```
python -m streamlit run app.py      # dashboard at http://localhost:8501  
make train SIGNAL=vibration         # train on real data (after §7 ingest)
```

That's everything. For the *why* behind each stage, read README.md and docs/build-walkthrough.md; for defense preparation see docs/defense-study-guide.md.